Informatics

The R language and system
with programming examples
Data Structures

Claudio Sartori
Department of Computer Science and Engineering
claudio.sartori@unibo.it
https://www.unibo.it/sitoweb/claudio.sartori/

matrix

- a two-dimensional vector
- all the elements are of the same type

```
> mat <-matrix(
            data=c(2,3,5,7,11,13,17,19,23,29,31,37),
            nrow=3,ncol=4)
> mat
     [,1] [,2] [,3] [,4]
[1,]    2    7   17   29
[2,]    3   11   19   31
[3,]    5   13   23   37
```

matrix – filling options

- the number of given values can be shorter than necessary, if with an *exact repetition* it is possible to fill the matrix
- in particular, if we give exactly nrows or ncols values, the matrix is *filled* as necessary

```
> mat <-matrix(data=c(2,3,5,7),nrow=3,ncol=4)
> mat
     [,1] [,2] [,3] [,4]
[1,]    2    7    5    3
[2,]    3    2    7    5
[3,]    5    3    2    7
```

Algorithm: matrix multiplication

- parameters
  - A, B, matrices to multiply
- use
  - m, p, n: relevant dimensions of matrices
  - i, j, k: indexes to drive loops
  - C: product matrix
- extract number of rows and columns from the two matrices passed as parameters
- if the dimensions are not compatible return NULL
- create C as an empty double vector of dimension m*p
- repeat for each row i of A
  - repeat for each column j of B
    - initialize to zero element i,j of C
    - repeat for each column k of A
      - add A[i,k]*B[k,j] to C[i,j]
- return C

```
matProd <- function (A, B) {
  m <- dim(A)[1] # extracts number of rows of A
  p <- dim(B)[2] # extracts number of columns of B
  n <- dim(A)[2] # extracts number of columns of A
  if (dim(B)[1] != n) # compare n with number of rows of B
    return(NULL) # if dimensions not compatible return NULL
  C <- vector(mode = "double", length = m * p)
  dim(C) <- c(m,p)
  for (i in seq_len(m)) # for each row of A
    for (j in seq_len(p)) { # for each column of B
      C[i,j] <- 0 # initialize the summation
      for (k in seq_len(n))
        # compute the summation
        C[i,j] <- C[i,j] + A[i,k] * B[k,j]
    }
  return(C)
}
```

array

- generalization of matrix: any number of *dimensions*
- all the elements are of the same type

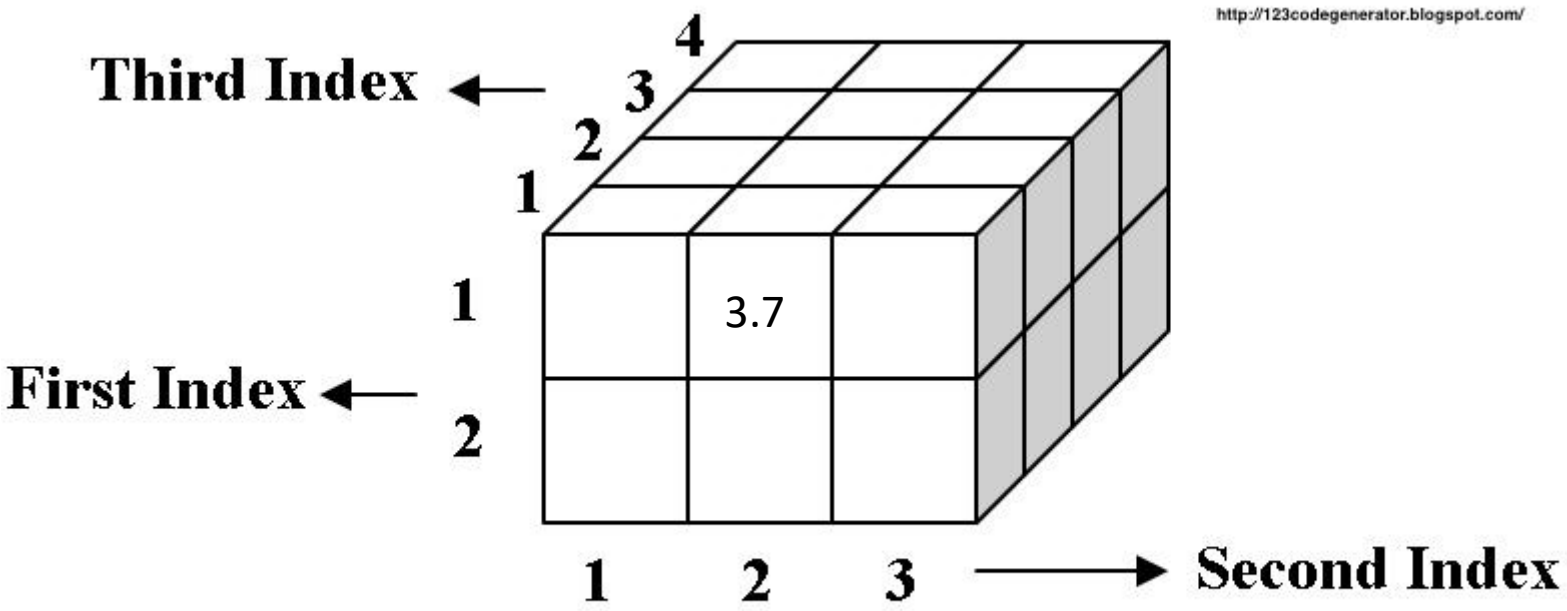
```
> arr <-array(
   data=c(2,3,5,7,11,13,17,19,23,29,31,37),
   dim=c(2,3,2))
> arr
, , 1

     [,1] [,2] [,3]
[1,]    2    5   11
[2,]    3    7   13

, , 2

     [,1] [,2] [,3]
[1,]   17   23   31
[2,]   19   29   37
```

Data structures and mathematics

- A vector of n doubles can represent a point in an n-dimensional space
  - $x \in R^n$
- A matrix of doubles with m rows and n columns can represent m points in an n-dimensional space
  - $x_i \in R^n, i = 1, \dots m$
- An array of numbers with n-dimensions can represent values associated to points in an n-dimensional discrete space

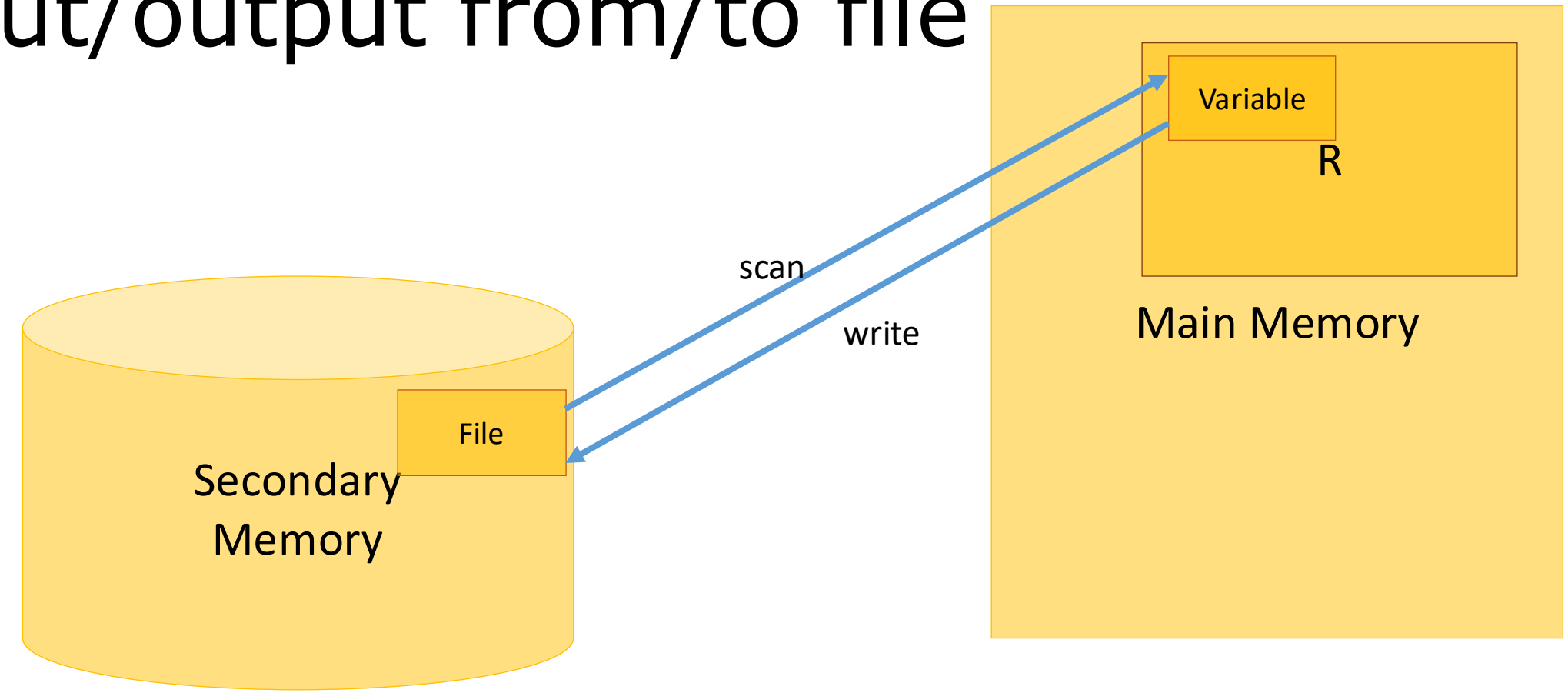
Use of arrays

a[1,2,1]<- 3.7



Three-dimensional array with twenty four elements

Input/output from/to file



Input from file – need complex type

```
> v <- scan("text1.txt")# by default reads double
Read 6 items
> print (v)
[1] 2 4 7 3 8 4
> dim(v) <- c(2,3) # reshaping
> print (v)
     [,1] [,2] [,3]
[1,]    2    7    8
[2,]    4    3    4
```

text1.txt

```
2
4
7
3
8
4
```

data.frame

Example

- When doing a market research survey you often have questions such as:
  - 'Are your married?' or 'yes/no' questions (= Boolean data type)
  - 'How old are you?' (= numeric data type)
  - 'What is your opinion on this product?' or other 'open-ended' questions (= character data type)
  - …
- The respondents' answers to the questions formulated above, is a data set of different data types.
- A data frame has the variables of a data set as columns and the observations as rows.

data frame

one of the example data built-in in R

Row names

Column names

- in essence: something like a vector of homogeneous lists, or a list of homogeneous vectors

```
> mtcars
                   mpg cyl  disp  hp drat    wt  qsec vs am gear carb
Mazda RX4         21.0   6 160.0 110 3.90 2.620 16.46  0  1    4    4
Mazda RX4 Wag     21.0   6 160.0 110 3.90 2.875 17.02  0  1    4    4
Datsun 710        22.8   4 108.0  93 3.85 2.320 18.61  1  1    4    1
...
Maserati Bora     15.0   8 301.0 335 3.54 3.570 14.60  0  1    5    8
Volvo 142E        21.4   4 121.0 109 4.11 2.780 18.60  1  1    4    2
>
```

data.frame - accessing

- head()
  - shows the first (last) n rows of the data frame
- tail()
- row and column index
- row and/or column name
- an empty index means all the row/column

data.frame –accessing (ii)

```
> mtcars$disp
 [1] 160.0 160.0 108.0 258.0 360.0 225.0 360.0 146.7 140.8 167.6 167.6 275.8
[13] 275.8 275.8 472.0 460.0 440.0  78.7  75.7  71.1 120.1 318.0 304.0 350.0
[25] 400.0  79.0 120.3  95.1 351.0 145.0 301.0 121.0
> mtcars["Pontiac Firebird",]
                  mpg cyl disp  hp drat    wt  qsec vs am gear carb
Pontiac Firebird 19.2   8  400 175 3.08 3.845 17.05  0  0    3    2
> mtcars[1,1]
[1] 21
> mtcars[1,]
          mpg cyl disp  hp drat   wt  qsec vs am gear carb
Mazda RX4  21   6  160 110  3.9 2.62 16.46  0  1    4    4
> mtcars[,1]
 [1] 21.0 21.0 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 17.8 16.4 17.3 15.2
[15] 10.4 10.4 14.7 32.4 30.4 33.9 21.5 15.5 15.2 13.3 19.2 27.3 26.0 30.4
[29] 15.8 19.7 15.0 21.4
```

data.frame - creating

```
persons <- data.frame(name = c("John","Mary","Sue"),
                      phone = c("335 234 5678",
                                "338 876 5432",
                                "340 1234 567"),
                      birthdate = c(as.Date("1990/12/16"),
                                    as.Date("1988/11/10"),
                                    as.Date("1991/08/26")
                                    ),
                      weight = c(78,54,58)
)
```

data.frame - viewing

```
> print(persons)
  name          phone  birthdate weight
1 John 335 234 5678 1990-12-16     78
2 Mary 338 876 5432 1988-11-10     54
3  Sue 340 1234 567 1991-08-26     58
```

Input from file – data.frame

```
> gdp2014 <- read.csv("gdp2014.csv", sep=";",
                      stringsAsFactors = FALSE)
> head(gdp2014)
  country_code ranking        economy gdp_millions_usd
1          USA       1  United States         17419000
2          CHN       2          China         10354832
3          JPN       3          Japan          4601461
4          DEU       4        Germany          3868291
5          GBR       5 United Kingdom          2988893
6          FRA       6         France          2829192
```

Reshaping a data.frame

The values of first column become the names of the rows

```
> gdp2014 <- read.csv("gdp2014.csv",sep=";")
> row.names(gdp2014) <- gdp2014[,1]
> gdp2014[,1] <- NULL
> head(gdp2014)
    ranking        economy gdp_millions_usd
USA       1  United States         17419000
CHN       2          China         10354832
JPN       3          Japan          4601461
DEU       4        Germany          3868291
GBR       5 United Kingdom          2988893
FRA       6         France          2829192
```

Eliminate the first column

data frame – accessing (iii)

```
> gdp2014["USA",] # access by row name
    ranking        economy gdp_millions_usdUSA
1 United States              17419000
> gdp2014[1,] # access by row number
    ranking        economy gdp_millions_usdUSA
1 United States              17419000
```

Summary of data.frame functions (1)

- **`row.names / col.names`**
  - get or set the rows/columns names
- **`nrow / ncol`**
  - get number of rows / columns
- **`dim`**
  - get or set a vector with number of rows and columns
- **`str`**
  - report on the structure of the dataframe

Summary of data.frame functions (2)

- **`head / tail`**
  - preview on the beginning/ending rows
- **`summary`**
  - descriptive statistics on the columns
- **`View`**
  - opens a window with tabular view

Summary of data.frame functions (3)

- **`read.csv("text.txt")`**
  - read from a text file in secondary memory
  - header = True/False
    - if True the first row of the file is used as column names
  - sep [default is ","]
    - separator between values in a row
  - row.names/col.names
    - set the properties by passing an appropriate vector of names, otherwise defaults are assumed
    - default row names are numbers
    - default column names are V followed by column number

Summary of data.frame functions (4)

- **`write.csv(df, “text.txt”)`**
  - write into a text file in secondary memory the dataframe df
  - header = True/False
    - if True the first row of the file is used as column names
  - row.names = True/False
    - if True a column of row names is written
  - sep [default is ”,”]
    - separator between values in a row

Summary of data.frame functions (5)

- **`cbind / rbind`**
  - add columns / rows
- **`df$column_name <- NULL`**
  - delete column by name from **`df`**
- **`subset(df, <boolean_vector>)`**
  - keeps only the rows corresponding to TRUE

Examples

- given a data frame remove duplicated rows
- given two data frames with the same structure and without duplicates
  - produce the intersection
  - produce the union
- see **`example029_dataframe_set_ops.zip`**

Intersection

df1	i	df2	j	keep
G		G	*	T
W	*	S		F
F		W		T
H		F		T

Join and exploratory data analysis

- Show example RPubs - Cars Dataset
- Explain the concept of join
- Explain why and how to do it
- discuss the example

Data Types: Factors

Factor variables in R

- *categorical variables* that can be either numeric or string
- a number of advantages to converting categorical variables to factors
- used in statistical modeling where they will be implemented correctly
  - assign the correct number of degrees of freedom
- storing string variables as factor variables is a more efficient use of memory
- **`factor`** function
- the only required argument is a vector of values that can be either string or numeric
- optional arguments include
  - levels
    - determines the categories of the factor variable
    - the default is the sorted list of all the distinct values of the data vector
  - ordered = FALSE
    - logical, if true the levels are assumed as ordered
  - labels
    - a vector of values that will be the labels of the categories in the levels argument
  - exclude
    - which levels will be classified as NA in any output using the factor variable

Factor

- symbolic representation of the values of a *categorical variable*
- the values are called *levels*
- Example: air quality, values
  - good
  - fair
  - sufficient
  - poor
  - bad
  - dangerous

Ordering of factor values

- when you create a factor you can decide
  ordered = FALSE or
  ordered = TRUE
- in the first case there not exists any order relationship among the values
- in the second case there exists an order relationship between any pair of values
- independently form the “ordered” property, we can request an ordered output, only for presentation purposes, not for computations

Pros and cons of factors

PRO

- when the number of distinct values in the column is much smaller than the number of rows
  - coding saves space (mildly important)
- when we need an ordering that is different from the alphabetic order given by the strings
- when we want to prevent ordering
- when we need to build cross tabulations based on that column

- CON
- when the number of distinct values is very high
- when it is unlikely to need cross tabulations based on that column
- when the alphabetic ordering is useful, but it is enough

R example

```
# without factor
air.quality.last.week <-
   c("good","fair","fair","sufficient",
     "fair","fair", "good")
print(air.quality.last.week)
[1] "good" "fair" "fair" "sufficient" "fair" "fair"
[7] "good"
```

R example

```
# with factor
air.quality.last.week.f <-
 factor(c("good","fair","fair","sufficient","fair",
          "fair", "good"),
        levels = c("good", "fair", "sufficient",
                   "poor", "bad"),
        ordered = TRUE)
print(air.quality.last.week.f)
[1] good fair fair sufficient fair fair good
Levels: good < fair < sufficient < poor < bad
```

Using factors with data frames

- When you create a data frame or read it from a file, there is a default value

  `stringsAsFactors = TRUE`

- this will convert all the columns of type **`string`** into **`factors`**
- this means that:
  - those columns will be coded and stored as integers, but the coded values will be decoded to the original value for any display purpose
  - the coded values are stored as integers, but they *aren't integers*, in the sense that arithmetic operations are not allowed

Operators allowed for factor values

Operators	Ordered Factor	Non Ordered Factor
+ - * / ^	no	no
== !=	yes	yes
>= > < <=	yes	no

Some numeric computations

$$n! = \begin{cases} 1 & \text{if } n = 0 \text{ or } n = 1 \\ n \cdot (n-1)! & \text{if } n > 1 \end{cases}$$

$$F(n) = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F(n-1) + F(n-2) & \text{if } n > 1 \end{cases}$$